

Docker & Podman Cheat Sheet

Run apps in containers. Podman takes the same commands: just type podman instead of docker.

Using it

```
docker run -d --name web -p 8080:80 nginx:1.27 ← nginx in the background; host port 8080 → container 80
docker run --rm -it debian:12 bash ← a throwaway shell; the container is deleted on exit
```

run option	Means	run option	Means
-d	Detached: run in the background	-v /srv/web:/data	Bind mount a host folder
--name web	Name it (else you get a random one)	-v webdata:/data	Named volume
-p 8080:80	Publish port: host : container	-e TZ=UTC	Set an env variable
-it	Interactive terminal (for shells)	--restart unless-stopped	Restart after crash / reboot
--rm	Delete the container when it exits	--network mynet	Join a network you made

Everyday commands (docker ...)

Command	Does	Command	Does
ps / ps -a	Running / all containers	images	Images on this machine
logs -f web	Follow its output (--tail 50)	pull nginx:1.27	Download an image
exec -it web sh	Shell inside a running container	rmi nginx:1.27	Delete an image
stop / start web	Stop (SIGTERM, then kill) / start	build -t app:1 .	Build from ./Dockerfile
rm web	Delete a stopped one (-f : force)	volume ls	Volumes (also network ls)
inspect web	Full JSON: IPs, mounts, env...	system df	Disk used by images, volumes...
stats	Live CPU / memory per container	system prune	Deletes all stopped containers

Careful: **system prune -a** also removes every image not in use; **--volumes** also unused volumes (your data).

Reading the output of docker ps -a

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
3f2a9c1d7b4e	nginx:1.27	"/docker-entrypoint..."	2 days ago	Up 2 days	0.0.0.0:8080->80/tcp	web
8b1e44c0a9d2	postgres:16	"docker-entrypoint.s..."	2 days ago	Up 2 days (healthy)	5432/tcp	db
c71d0e5f3a18	debian:12	"bash"	3 weeks ago	Exited (137) 3 weeks ago		oldest

Column	Means
CONTAINER ID / NAMES	Either one works in commands (the first few ID letters are enough)
STATUS	Up = running · Exited (0) = finished OK · (1) = error, read its logs · (137) = killed
0.0.0.0:8080->80/tcp	Published: host port 8080 (all addresses) → container port 80
5432/tcp	Open only to other containers, not reachable from the network

Compose: a whole app in one file (compose.yaml)

services:	Command (run in its folder)	Does
web:	docker compose up -d	Creates / updates and starts all
image: nginx:1.27	docker compose ps	Status of this project's containers
ports: ["8080:80"]	docker compose logs -f web	Follows one service's logs
volumes:	docker compose pull	Downloads newer images; then up -d
- webdata:/usr/share/nginx/html	docker compose exec web sh	Shell inside a running service
restart: unless-stopped	docker compose down	Removes containers (volumes kept)
volumes:		
webdata:		

Podman

Point	Detail	Point	Detail
Near drop-in	alias docker=podman works for most use	No daemon	Containers are plain processes
Rootless	Runs as you; ports under 1024 need root	Compose	podman compose (Podman 4.7+)
At boot	Quadlet (4.4+): a .container file for systemd	Old way	podman generate systemd (deprecated)

Remember: anything a container writes outside a volume or bind mount is **lost** when the container is removed, and updating to a new image means recreating it. Keep data in a volume (**-v**).